

Mask-Aware Shapley Attribution

1 Shapley Values for Cooperative Games

Consider a **coalitional game** with player set N where any subset (**coalition**) $S \subseteq N$ can form and achieve value given by a **characteristic function** (also called a **value function**) $v : 2^N \rightarrow \mathbb{R}$ with $v(\emptyset) = 0$. In the machine learning context we will refer to v as the **value function**. The problem is to find a fair **allocation** $\phi_i(v)$ for each player $i \in N$ that distributes $v(N)$ according to agreed fairness principles.

Four axioms define any fair **allocation mechanism**:

1. **Efficiency:** The axiom requires $\sum_{i \in N} \phi_i(v) = v(N)$. It ensures the full value of the player set is distributed.
2. **Symmetry:** The axiom states that if $v(S \cup \{i\}) = v(S \cup \{j\})$ for all S not containing i or j , then $\phi_i(v) = \phi_j(v)$. It enforces equal treatment of players with identical marginal contributions.
3. **Dummy Player:** The axiom states that if $v(S \cup \{i\}) = v(S)$ for all S not containing i , then $\phi_i(v) = 0$. It recognises that players who never contribute value receive none of the payoff.
4. **Linearity:** The axiom requires $\phi_i(av + bw) = a\phi_i(v) + b\phi_i(w)$ for any value functions v, w and scalars $a, b \in \mathbb{R}$. It ensures additivity across games and positive homogeneity with respect to scaling.

These four axioms uniquely determine the allocation. The **Shapley value** for player i is given by:

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [v(S \cup \{i\}) - v(S)] \quad (1)$$

Derivation of the weight $\frac{|S|!(|N| - |S| - 1)!}{|N|!}$: Consider all $|N|!$ possible arrival orders (**permutations**) of the N players. For player i to join exactly the coalition S , the players in S must arrive before i , and the remaining $|N| - |S| - 1$ players must arrive after i . There are $|S|!$ ways to order the players arriving before i , and $(|N| - |S| - 1)!$ ways to order those arriving after i . Thus, the number of permutations where i joins coalition S is $|S|!(|N| - |S| - 1)!$, and the

probability is $\frac{|S|!(|N|-|S|-1)!}{|N|!}$. Therefore the Shapley value equals the expected marginal contribution under a uniform random arrival order.

The term $v(S \cup \{i\}) - v(S)$ is player i 's **marginal contribution** when joining coalition S .

Equivalently, averaging over all **permutations** $\pi \in \Pi(N)$, where $\Pi(N)$ denotes the set of all permutations of N :

$$\phi_i(v) = \frac{1}{|N|!} \sum_{\pi \in \Pi(N)} [v(S_\pi^i \cup \{i\}) - v(S_\pi^i)] \quad (2)$$

where S_π^i is the set of players appearing before i in permutation π .

2 Machine Learning Feature Attribution as Shapley Values

2.1 Formal Setup

We now set up the general SHAP framework for feature attribution. Consider a **predictive model** $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and an input $x \in \mathbb{R}^n$ with prediction $f(x)$. The goal is to quantify how much each feature in x contributes to this prediction. We formulate this task as a Shapley value computation by treating each **original feature as a player** in a cooperative game. We index the features by the player set $N = \{1, \dots, n\}$, with feature i corresponding to coordinate x_i .

To connect the model f with coalitional games we introduce the **masked predictor** $f(x; S)$. For any subset (**coalition**) $S \subseteq N$ we interpret S as the set of features that are treated as known, and we define $f(x; S)$ to be the model prediction when only the coordinates in S are available and the remaining coordinates $N \setminus S$ are treated as missing. Notice that we have not yet specified how to compute $f(x; S)$ in practice. This is a **major modeling choice** when computing Shapley values, and we will discuss it in detail in the next section. The special case $f(x; \emptyset)$ is the prediction when no feature information is used, and we refer to this quantity as the **baseline prediction**. We therefore set

$$b = f(x; \emptyset).$$

Given this masked predictor we define the **value function** $v : 2^N \rightarrow \mathbb{R}$ by

$$v(S) = f(x; S) - f(x; \emptyset)$$

for every coalition $S \subseteq N$. By construction, $v(\emptyset) = 0$ and $v(N) = f(x) - f(x; \emptyset)$, so the total game value equals the difference between the actual prediction and the baseline prediction.

A **Shapley explanation** of f at input x is a vector of feature attributions $\phi(x) = (\phi_1(x), \dots, \phi_n(x))$ associated with the game (N, v) . The SHAP framework enforces the **efficiency** condition

$$v(N) = f(x) - f(x; \emptyset) = \sum_{i=1}^n \phi_i(x),$$

so that the total attribution exactly matches the game value $v(N)$. Each attribution $\phi_i(x)$ is defined to be the Shapley value of feature i in the cooperative game (N, v) . Formally, writing $n = |N|$, the Shapley value for feature i is

$$\phi_i(x) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! (n - |S| - 1)!}{n!} [v(S \cup \{i\}) - v(S)].$$

This expression averages feature i 's **marginal contribution** $v(S \cup \{i\}) - v(S)$ over all coalitions S that do not contain i , with weights corresponding to the fraction of permutations in which S precedes i . Equivalently, let $\Pi(N)$ denote the set of all permutations of N and let S_π^i be the set of features that appear before i in permutation π . Then we can write

$$\phi_i(x) = \frac{1}{n!} \sum_{\pi \in \Pi(N)} [v(S_\pi^i \cup \{i\}) - v(S_\pi^i)]$$

and, using the definition of v , this becomes

$$\phi_i(x) = \frac{1}{n!} \sum_{\pi \in \Pi(N)} [f(x; S_\pi^i \cup \{i\}) - f(x; S_\pi^i)].$$

Thus, in the SHAP setting the Shapley values admit two complementary interpretations. First, they **allocate the prediction above the baseline** $f(x; \emptyset)$ so that the attributions sum to $f(x) - f(x; \emptyset)$. Second, each $\phi_i(x)$ equals the **expected marginal change in prediction** when feature i is added to the set of previously included features in a uniformly random ordering.

2.2 Intuition: Toy House Price Example

Suppose we observe five houses, each described by three features—lot size (acres), total square footage, and number of rooms—and a sale price y in dollars, as shown in Table 1.

House	Acres	Square feet	Rooms	Price (\$)
1	0.10	1200	3	220,000
2	0.15	1400	3	260,000
3	0.20	1600	4	300,000
4	0.25	1800	4	340,000
5	0.30	2000	5	380,000

Table 1: Toy dataset of five houses with simple features and sale prices.

Let $x \in \mathbb{R}^3$ collect the three features for a house and let $f(x)$ be a trained regression model's predicted sale price. For a particular house x , for example $x_5 = (0.30, 2000, 5)$, the feature attribution problem is to determine how each coordinate of x (acres, square footage, rooms) contributes to the prediction

$f(x)$. Assume for concreteness that the model outputs $f(x_5) = 360,000$ dollars for house 5.

In the notation of the formal setup, we consider the extended prediction operator $f(x_5; S)$ for coalitions $S \subseteq N = \{1, 2, 3\}$, where the three features correspond to acres, square footage, and rooms. We define the **baseline prediction** to be $f(x_5; \emptyset)$, which in this example we take to be the average sale price across the five houses. If the average price is 300,000 dollars, then $f(x_5) - f(x_5; \emptyset) = 60,000$ dollars. This quantity is the total value $v(N)$ that must be allocated across the three features by the Shapley attributions. The feature attributions decompose the 60,000-dollar improvement above the baseline. Equivalently, we can interpret each $\phi_i(x_5)$ as the expected marginal change in the prediction when feature i is added to a coalition of previously included features in a uniformly random order. Concretely, we start from the baseline prediction $f(x_5; \emptyset)$, choose a random ordering of the three features, and then add them one at a time, recording the increment $f(x_5; S \cup \{i\}) - f(x_5; S)$ each time feature i is added to a set S of features that appear earlier in the ordering. The Shapley value $\phi_i(x_5)$ is the average of this marginal change across all six possible orderings.

To make this explicit, label acres, square footage, and rooms as features 1, 2, 3 so that $N = \{1, 2, 3\}$. For feature 1, the six permutations of N are

$$(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1).$$

In each permutation π , let S_π^1 be the set of features that appear before 1. The marginal contribution of feature 1 in permutation π is

$$\Delta_1^\pi = f(x_5; S_\pi^1 \cup \{1\}) - f(x_5; S_\pi^1).$$

For example, in permutation $(1, 2, 3)$ we have $S_{(1,2,3)}^1 = \emptyset$ and

$$\Delta_1^{(1,2,3)} = f(x_5; \{1\}) - f(x_5; \emptyset),$$

while in permutation $(2, 1, 3)$ we have $S_{(2,1,3)}^1 = \{2\}$ and

$$\Delta_1^{(2,1,3)} = f(x_5; \{1, 2\}) - f(x_5; \{2\}).$$

The Shapley value for feature 1 is then

$$\phi_1(x_5) = \frac{1}{6} \sum_{\pi \in \Pi(\{1,2,3\})} \Delta_1^\pi.$$

Analogous expressions hold for features 2 and 3, obtained by averaging their marginal contributions over the same six permutations.

3 Defining the Masked Predictor

In the previous section, we defined the value function $v(S) = f(x; S) - f(x; \emptyset)$ and introduced the **baseline prediction** $f(x; \emptyset)$. We did not yet specify how

the **masked predictions** $f(x; S)$ are computed. The predictive model f is usually defined only on full feature vectors x , so we must decide how to evaluate the **masked predictor** $f(x; S)$ when only a subset of features is treated as known.

Shapley explanations depend critically on how we define the masked predictor because it is central to the value function:

1. **Baseline prediction:** For a fixed input x , the baseline prediction $f(x; \emptyset)$ determines the total quantity $f(x) - f(x; \emptyset)$ that the Shapley values $\phi_i(x)$ must allocate. The value function $v(S) = f(x; S) - f(x; \emptyset)$ depends directly on the chosen baseline.
2. **Masked predictor semantics:** The masked predictions $f(x; S)$ specify how this total value is distributed across coalitions S . Different definitions of $f(x; S)$ induce different value functions $v(S)$ and thus different Shapley explanations, even when the underlying model f and the input x are fixed.
3. **Faithfulness to the original model:** The Shapley game is built on the masked predictor $S \mapsto f(x; S)$ rather than directly on the original predictor $x \mapsto f(x)$. If $f(x; S)$ is defined using unrealistic hybrids or an inappropriate background distribution, the resulting value function $v(S)$ can behave very differently from f on typical inputs, and the Shapley attributions may fail to faithfully reflect how f actually uses its features.

Several standard choices exist for defining $f(x; S)$. Intuitively, the **masked predictor** $f(x; S)$ represents the prediction when features in S are treated as known and features in $N \setminus S$ are treated as unknown. We evaluate $f(x; S)$ by applying f to a full feature vector in which coordinates in S are fixed to x_S and coordinates in $N \setminus S$ are filled in according to a chosen rule. Several standard choices for this rule appear in the SHAP literature:

1. **Fixed baseline substitution:** Choose a baseline input $x^{(0)} \in \mathbb{R}^n$ (for example, a zero vector or a reference input) and define a hybrid input that uses x_S on coordinates in S and $x_{N \setminus S}^{(0)}$ on coordinates in $N \setminus S$. The masked predictor is $f(x; S) = f(x_S, x_{N \setminus S}^{(0)})$. Unknown features are always replaced with fixed baseline values, which is simple but can evaluate f on out-of-distribution inputs. Formally, for each coalition $S \subseteq N$ we define a hybrid input $x^{(S)} \in \mathbb{R}^n$ by

$$x_i^{(S)} = \begin{cases} x_i, & i \in S, \\ x_i^{(0)}, & i \notin S, \end{cases}$$

so that $x^{(S)} = (x_S, x_{N \setminus S}^{(0)})$ and

$$f(x; S) := f(x^{(S)}).$$

In words, $x^{(S)}$ agrees with the original input x on coordinates in S and with the baseline input $x^{(0)}$ on coordinates in $N \setminus S$.

2. **Mean imputation:** Let $\bar{x}_{N \setminus S} = \mathbb{E}[X_{N \setminus S}]$ denote the mean of the unknown features under the training distribution. The masked predictor is $f(x; S) = f(x_S, \bar{x}_{N \setminus S})$. Unknown features are replaced with their mean training values, which approximates averaging over the marginal distribution using only its first moment.
3. **Marginal expectation (interventional semantics):** Let $P(X)$ denote the joint training distribution over features. The masked predictor is defined by the *marginal expectation*

$$f(x; S) = \mathbb{E}_{X_{N \setminus S} \sim P(X_{N \setminus S})} [f(x_S, X_{N \setminus S})].$$

Unknown features are sampled from their marginal distribution, so conditioning on x_S does not affect their distribution. In practice, this expectation is approximated using samples from the empirical training distribution.

4. **Conditional expectation (conditional semantics):** Instead of breaking feature dependencies, we can preserve them by averaging over the conditional distribution of the unknown features given the known ones. The masked predictor is

$$f(x; S) = \mathbb{E}_{X_{N \setminus S} \sim P(X_{N \setminus S} | X_S = x_S)} [f(x_S, X_{N \setminus S})].$$

Unknown features follow their conditional distribution given $X_S = x_S$, which properly accounts for feature correlations but requires modeling $P(X_{N \setminus S} | X_S)$ in high dimensions.

A mask-aware approach—detailed in the closing section—replaces these ad hoc choices with a learned semantics of missingness. By exposing the model to random masks during training, we define $f(x; S)$ directly through a mask channel rather than through hybrids built from external backgrounds. This keeps $f(x; S)$ on-manifold, makes $f(x; \emptyset)$ an in-model baseline, and gives access to the conditional behaviour the Shapley value requires without separately estimating $P(X_{N \setminus S} | X_S)$.

From a prediction point of view, the **conditional expectation** definition yields the most faithful notion of prediction when only a subset of features is known. Let X denote the random input vector with distribution $P(X)$ and consider predicting the scalar response $f(X)$ under squared error loss when only X_S is observed. The prediction that minimises the expected squared error given $X_S = x_S$ is

$$g_S(x_S) = \mathbb{E}[f(X) | X_S = x_S],$$

which is exactly the conditional masked predictor obtained by averaging $f(x_S, X_{N \setminus S})$ over the conditional distribution of the unknown coordinates $X_{N \setminus S}$ given $X_S = x_S$. In words, conditional expectation computes the best mean-squared-error prediction for $f(X)$ using all information in the observed features X_S while sampling the unknown features in a way that respects their empirical dependencies with X_S . By contrast, marginal expectation ignores these dependencies

by treating $X_{N \setminus S}$ as independent of X_S , and fixed-baseline or mean imputation correspond to degenerate choices of $P(X_{N \setminus S} \mid X_S)$ that are no longer optimal given X_S in this squared-error sense. Thus, when the goal is to explain predictions under the observed data distribution and to treat known and unknown coordinates coherently, conditional expectation provides a principled notion of “best prediction given x_S ” for defining the masked predictor.

This perspective also clarifies why simple **zero** or **mean** baselines can produce unfaithful Shapley games. A zero baseline sets $x^{(0)}$ to the all-zero vector and evaluates hybrids $f(x_S, x_{N \setminus S}^{(0)})$ that may lie far outside the support of the training distribution, so the value function $v(S)$ reflects model behaviour on highly unrealistic inputs rather than on typical data. Similarly, a mean baseline or mean imputation uses a single fixed vector or the marginal mean $\bar{x}_{N \setminus S}$ for all coalitions S , which discards the dependence between X_S and $X_{N \setminus S}$ and does not approximate the conditional expectation $\mathbb{E}[f(X) \mid X_S = x_S]$. In both cases, the masked predictor $S \mapsto f(x; S)$ is anchored to off-manifold or weakly informative reference points, so the induced value function $v(S)$ can deviate substantially from how f behaves on realistic inputs. As a result, Shapley attributions built on zero or mean baselines may emphasise artefacts of the chosen reference rather than faithfully capturing how the original model uses its features across the data distribution.

4 Computational Challenges

Computing exact Shapley feature attributions for a model prediction $f(x)$ can be expressed using the permutation-based definition

$$\phi_i(x) = \frac{1}{n!} \sum_{\pi \in \Pi(N)} \left[f(x; S_\pi^i \cup \{i\}) - f(x; S_\pi^i) \right],$$

where $\Pi(N)$ is the set of all permutations of the feature index set $N = \{1, \dots, n\}$ and S_π^i denotes the set of features that appear before i in permutation π . (Equivalently, one can work with the coalition-based expression from the Shapley preliminaries, but the permutation view is more convenient for discussing practical algorithms that add features sequentially.)

This leads to two fundamental computational bottlenecks.

4.1 Summation Over Permutations

The first bottleneck is *combinatorial*. The permutation-based Shapley formula involves $n!$ terms for each feature i . Even if each evaluation of the masked predictor were extremely cheap, the factor of $n!$ quickly becomes prohibitive as n grows (e.g., $n > 20$). Approximate Shapley methods therefore replace the full sum over $\Pi(N)$ with approaches such as:

- Monte Carlo sampling of a limited number of permutations,

- restricting attention to local neighborhoods of features, or
- exploiting model structure (e.g., linearity, additivity, or neural network layers) to collapse the Shapley sum analytically.

4.2 Masked Predictor Evaluation

The second bottleneck concerns the cost of evaluating the masked predictor $f(x; S)$ itself. Each Shapley term requires computing

$$f(x; S_\pi^i) \quad \text{and} \quad f(x; S_\pi^i \cup \{i\})$$

for different coalitions S_π^i determined by the permutations. Let C_{mask} denote the cost of a single masked evaluation. In classical SHAP semantics this cost ranges from:

- a *single forward pass* of the original model f when missing features are replaced by fixed baselines or means, to
- *multiple forward passes combined with Monte Carlo averaging* when missing features are sampled from a background distribution to approximate marginal expectations.

In our mask-aware setting, $f(x; S)$ is implemented directly via a mask channel, so computing $f(x; S)$ amounts to an ordinary forward pass through the mask-aware network and C_{mask} is comparable to the cost of standard prediction.

Regardless of the semantics, the total cost of any Shapley procedure can be decomposed as

$$\text{cost} = (\text{number of masked evaluations}) \times C_{\text{mask}},$$

where the first factor is determined by how we approximate or traverse the $n!$ permutations, and the second factor is determined by how the masked predictor $f(x; S)$ is implemented. These two bottlenecks jointly motivate the approximation strategies and model-specific optimizations discussed next.

5 What is Done in Practice

The theoretical discussion above emphasised conditional expectations as the principled way to define the masked predictor

$$f(x; S) = \mathbb{E}[f(x_S, X_{N \setminus S}) \mid X_S = x_S],$$

but this is rarely implemented in everyday practice. Most widely used SHAP-style methods instead adopt simpler *marginal* or *baseline-based* semantics and focus on alleviating the two computational bottlenecks.

5.1 Background-Based Marginal Semantics

In popular toolkits such as SHAP and Captum, missing features are almost always handled via a *background dataset*. Concretely, one selects a small collection of background inputs

$$\{x^{(b)}\}_{b=1}^B,$$

typically drawn from or summarising the training data, and approximates masked predictions by forming hybrids between the point of interest x and these background samples:

- **Marginal / interventional semantics.** Missing coordinates are replaced by values taken from background examples, independently of x_S . In our notation this corresponds to the *marginal expectation*

$$f(x; S) \approx \frac{1}{B} \sum_{b=1}^B f(x_S, X_{N \setminus S}^{(b)}),$$

which treats the unknown features $X_{N \setminus S}$ as independent of the observed x_S . The SHAP library exposes this via background matrices for its masking functions and DeepExplainer, and Captum exposes it via baselines and background tensors in its PyTorch APIs.

- **Finite-background approximations.** Rather than integrating over the full marginal distribution, practitioners work with a finite background set and approximate the marginal expectation by averaging over a handful of hybrids, or by using the empirical mean of the background as a single reference input.
- **Fixed baselines.** In the simplest workflows, a single baseline is used for all coalitions, such as the all-zero vector, feature-wise means, or a hand-chosen reference example. This corresponds to fixed-baseline or mean-imputation semantics and eliminates any notion of sampling over a background distribution.

Across these practices, the conditional distribution $P(X_{N \setminus S} \mid X_S = x_S)$ is not modelled explicitly: masked features are filled in from an unconditional background, and feature dependencies are ignored by construction.

5.2 Permutation-Based Approximations in Software

Libraries such as Captum implement perturbation-based Shapley approximations that directly address the combinatorial bottleneck by *sampling* rather than enumerating permutations. For example, Shapley-value sampling methods draw random permutations of the input features, add them one by one to a baseline, and average the resulting marginal contributions over a limited number of sampled permutations. This reduces the summation-over-permutations cost from $O(n!)$ to $O(Kn)$ for K sampled permutations, at the expense of Monte Carlo variance, while still requiring masked evaluations for each sampled coalition.

5.3 DeepSHAP and Deep-Learning-Specific Approximations

For deep networks, a widely used state-of-practice approach is DeepSHAP, as implemented in the SHAP library (DeepExplainer) and in PyTorch via Captum’s DeepLiftShap. DeepSHAP builds on DeepLIFT and combines two ideas:

1. **Marginal semantics via background data.** A small set of background inputs $\{x^{(b)}\}_{b=1}^B$ is chosen, typically from the training distribution. These examples define an interventional background distribution for missing features. For each background point, the model is evaluated on $x^{(b)}$ and on the input of interest x , and differences are propagated layer by layer. Averaging attributions over the background set approximates Shapley values under marginal semantics.
2. **Layer-wise propagation of Shapley values.** Instead of explicitly summing over coalitions or sampling permutations, DeepSHAP uses DeepLIFT-style forward and backward rules that, for common neural network building blocks (linear layers, elementwise activations, certain pooling operations), coincide with or closely approximate the Shapley solution for those blocks. By composing these local rules along the computation graph, DeepSHAP analytically collapses the Shapley sum for each background point into a single forward and backward pass.

From the perspective of the two computational bottlenecks:

- DeepSHAP *addresses the summation-over-permutations bottleneck* by exploiting the compositional structure of neural networks and applying closed-form Shapley-style rules at the layer level. No explicit enumeration or sampling of permutations is required.
- DeepSHAP also *alleviates the masked-predictor bottleneck* relative to naive marginal Shapley: instead of computing $f(x; S)$ for many coalitions and many samples from a marginal distribution, it reuses a small background set and computes attributions via a small number of forward and backward passes, with overall cost on the order of B model evaluations.

However, like other background-based methods, DeepSHAP operates under marginal/interventional semantics and does not attempt to model the conditional distribution $P(X_{N \setminus S} \mid X_S = x_S)$. Its attributions therefore reflect how the model behaves when features are independently perturbed according to the chosen background, rather than how the model would behave under the ideal conditional expectation semantics discussed earlier.

These practical design choices—background-based marginal semantics, finite background sets, and network-structure-aware approximations such as DeepSHAP—define the current computational “sweet spot” for Shapley-style feature attribution in deep learning, and they form the baseline against which our mask-aware approach should be compared.

6 Mask-Aware Shapley Attribution

Motivated by the masking semantics and computational issues above, we now outline a mask-aware approach to Shapley-value feature attribution in which the model itself learns how to interpret missing features. The aim is to remove the dependence on arbitrary background distributions, keep all evaluations in-distribution, and make the notion of “feature absence” an explicit, learned part of the predictor.

6.1 Problem: Defining Feature Absence

Traditional Shapley methods (e.g., SHAP, Captum ShapleyValueSampling) rely on an external rule to decide what a “missing” feature means: zeros, training-set averages, or samples from a background distribution. Because typical models are trained only on fully observed inputs, masking features at attribution time produces off-manifold hybrids and makes the resulting Shapley values sensitive to the chosen background rather than to the model’s internal semantics. The consequence is that we do not evaluate the conditional distribution the model should follow when only a subset of features is observed.

6.2 Core Idea

Train a predictive model that is explicitly aware of feature masking, so that absence has a defined, learned meaning inside the model. Then compute Shapley values directly through this mask interface—without any external background distribution.

6.3 Proposed Solution

1. **Train with random feature masks.** During training, randomly mask subsets of input features (setting them to a fixed value or a special embedding) so the model learns to produce meaningful predictions under partial information:

$$x' = x \odot m, \quad m \sim \text{Bernoulli}(p).$$

2. **Learn conditional inference under missingness.** Because masked inputs appear during training, the model learns to approximate $f(x_S)$ when only subset S is available. The mask channel gives direct access to the conditional behaviour the Shapley game needs and lets us evaluate the conditional distribution that is actually desired.
3. **Compute Shapley values directly via masking.** During attribution, sample feature subsets S using the same masking mechanism and evaluate

$$\phi_i = \mathbb{E}_{S \subseteq N \setminus \{i\}} [f(x_{S \cup \{i\}}) - f(x_S)].$$

No background distribution is needed—the mask itself encodes absence. Every evaluation remains in-distribution because the model was trained to handle masked inputs.

4. **Results.** Attributions are causally meaningful (absence is a learned intervention), computation stays faithful to the Shapley definition, off-manifold artefacts disappear, and the same mask interface can support conditional or interventional variants of Shapley by varying the mask sampling strategy.